UNIVERSITY OF INFORMATION TECHNOLOGY
FACULTY OF INFORMATION SYSTEMS

Chapter 4

SQL

Dr. Cao Thi Nhan

Content

1. Introduction
2. SQL Data Definition and Data Types
3. INSERT, DELETE, and UPDATE Statements in SQL
4. Basic Retrieval Queries in SQL
5. More Complex SQL

Introduction

- SQL language
  - Considered one of the major reasons for the commercial success of relational databases
- **SQL**
  - **Structured Query Language**
  - Statements for data definitions, queries, and updates.
  - **Core specification**
  - Plus specialized **extensions**

SQL Data Definition and Data Types

- Terminology:
  - **Table**, **row**, and **column** used for relational model terms **relation, tuple**, and **attribute**
- **CREATE** statement
  - Main SQL command for data definition
- **SQL schema**
  - Identified by a **schema name**
  - Includes an **authorization identifier** and **descriptors** for each element
- Schema **elements** include
  - Tables, constraints, views, domains, and other constructs
- Each statement in SQL ends with a semicolon

SQL Data Definition and Data Types

- SQL **environment**
  - Installation of an SQL-compliant RDBMS on a computer system
- **CREATE SCHEMA** statement
  - **CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';**

CREATE TABLE command in SQL

- Specify a new relation
  - Provide name
  - Specify attributes and initial constraints
- Can optionally specify schema:
  - CREATE TABLE SchemaName.TableName …

    CREATE TABLE COMPANY.EMPLOYEE …

  Or
  - CREATE TABLE TableName …

    CREATE TABLE EMPLOYEE

Data types in Microsoft SQL Server

Data type	SQL Server
String	varchar(n), char(n), nvarchar(n), nchar(n), text, …
Number	tinyint, smallint, int, bigint, numeric(m, n), decimal(m,n), float, real, smallmoney, money
Date, time	smalldatetime, datetime…
Logical / Boolean	Bit (0, 1, NULL)
Others	Cursor, xml, spatial geometry types, binary strings (image, binary)

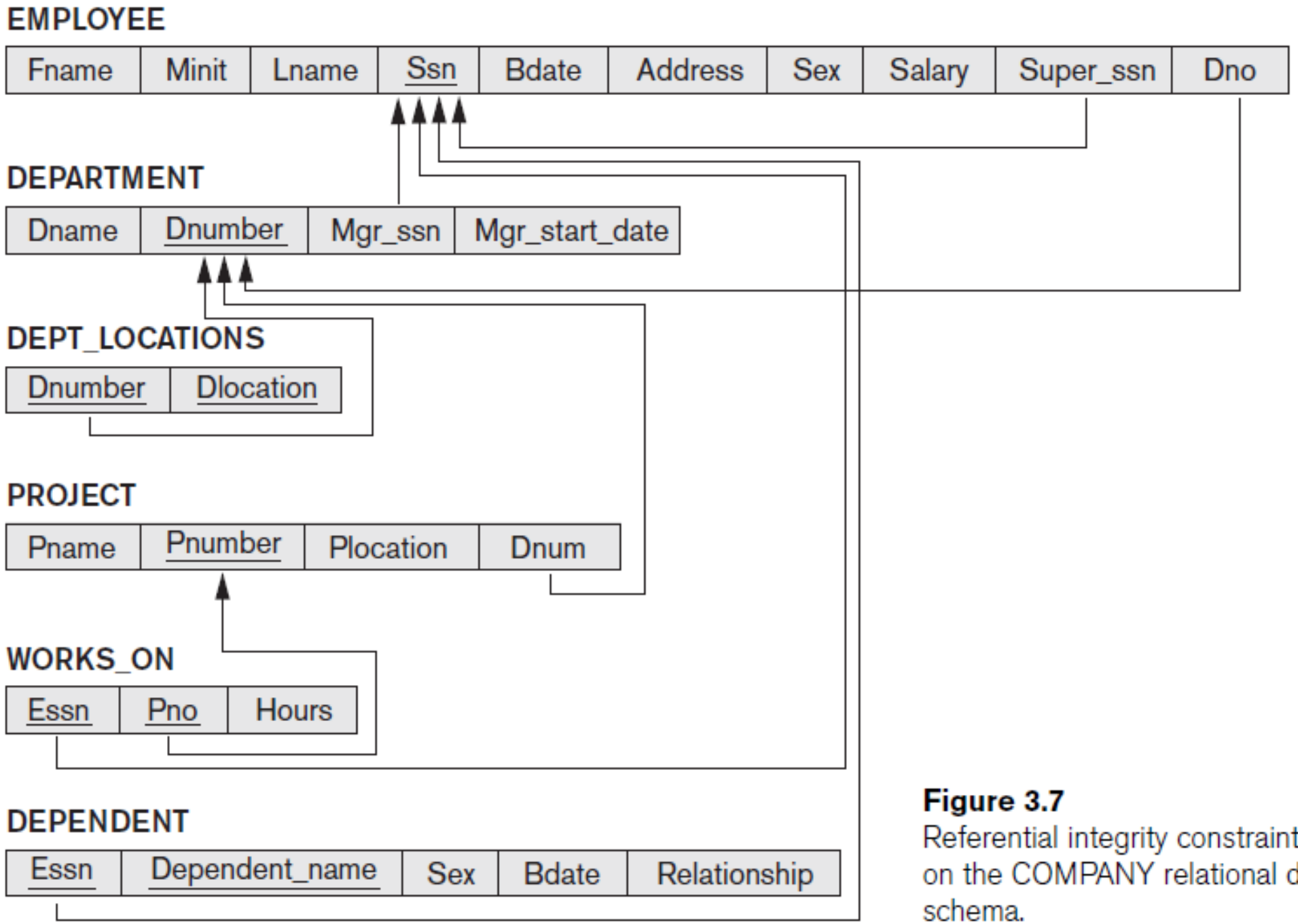

Figure 3.7
Referential integrity constraints displayed on the COMPANY relational database schema.

CREATE TABLE command in SQL

- **CREATE TABLE command**

Not null, Unique, Primary key, Foreign key …

```
CREATE TABLE <Table_name> (
    <Column name 1> <Data type> [<Constraint>],
    <Column name 2> <Data type> [<Constraint>],
    …
    [< Constraint >]
)
```

Primary key, Foreign key …

CREATE TABLE command in SQL

EMPLOYEE (Ssn, Fname, Minit, Lname, Bdate, Address, Sex, Salary, Super_ssn, Dno)

```
CREATE TABLE EMPLOYEE (
    Ssn          char(10) primary key,
    Fname        varchar(30) not null,
    Minit        varchar(2) not null,
    Lname        varchar(30) not null,
    Bdate        smalldatetime,
    Address      varchar(50),
    Sex          bit,
    Super_ssn    char(10),
    Dno          int
)
```

CREATE TABLE command in SQL

EMPLOYEE (Ssn, Fname, Minit, Lname, Bdate, Address, Sex, Salary, Super_ssn, Dno)

DEPARTMENT (Dnumber, Dname, Mgr_ssn, Mgr_start_date)

```
CREATE TABLE DEPARTMENT (
    Dnumber          int primary key,
    Dname            varchar(30) not null,
    Mgr_ssn          char(10) foreign key references EMPLOYEE (Ssn),
    Mgr_start_date   smalldatetime,
    )
```

?! Order

CREATE TABLE command in SQL

EMPLOYEE (Ssn, Fname, Minit, Lname, Bdate, Address, Sex, Salary, Super_ssn, Dno)

PROJECT (Pnumber, Pname, Plocation, Dnum)

WORKS_ON (Essn, Pno, Hours)

```
CREATE TABLE WORKS_ON (
  Essn        char(10),
  Pno         int,
  Hours       int not null,
  primary key (Essn, Pno)
  foreign key (Essn) references EMPLOYEE (Ssn),
  foreign key (Pno) references PROJECT (Pnumber),
)
```

?!

Primay key has some attributes

CREATE TABLE command in SQL


```
CREATE TABLE EMPLOYEE
        ( Fname          VARCHAR(15)        NOT NULL,
          Minit          CHAR,
          Lname          VARCHAR(15)        NOT NULL,
          Ssn            CHAR(9)            NOT NULL,
          Bdate          DATE,
          Address        VARCHAR(30),
          Sex            CHAR,
          Salary         DECIMAL(10,2),
          Super_ssn      CHAR(9),
          Dno            INT                NOT NULL,
        PRIMARY KEY (Ssn),
        FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn),
        FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE DEPARTMENT
        ( Dname          VARCHAR(15)        NOT NULL,
          Dnumber        INT                NOT NULL,
          Mgr_ssn        CHAR(9)            NOT NULL,
          Mgr_start_date DATE,
        PRIMARY KEY (Dnumber),
        UNIQUE (Dname),
        FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
```

Figure 4.1
SQL CREATE TABLE data definition statements for defining the COMPANY schema from Figure 3.7.

```
CREATE TABLE DEPT_LOCATIONS
        ( Dnumber          INT            NOT NULL,
          Dlocation        VARCHAR(15)    NOT NULL,
        PRIMARY KEY (Dnumber, Dlocation),
        FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE PROJECT
        ( Pname            VARCHAR(15)    NOT NULL,
          Pnumber          INT            NOT NULL,
          Plocation        VARCHAR(15),
          Dnum             INT            NOT NULL,
        PRIMARY KEY (Pnumber),
        UNIQUE (Pname),
        FOREIGN KEY (Dnum) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE WORKS_ON
        ( Essn             CHAR(9)        NOT NULL,
          Pno              INT            NOT NULL,
          Hours            DECIMAL(3,1)   NOT NULL,
        PRIMARY KEY (Essn, Pno),
        FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn),
        FOREIGN KEY (Pno) REFERENCES PROJECT(Pnumber) );
CREATE TABLE DEPENDENT
        ( Essn             CHAR(9)        NOT NULL,
          Dependent_name   VARCHAR(15)    NOT NULL,
          Sex              CHAR,
          Bdate            DATE,
          Relationship     VARCHAR(8),
        PRIMARY KEY (Essn, Dependent_name),
        FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn) );
```

Figure 4.1
SQL CREATE TABLE data definition statements for defining the COMPANY schema from Figure 3.7.

Relational data model example

1. **EMPLOYEE** (Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)
2. **DEPARTMENT** (Dnumber, Dname, Mgr_ssn, Mgr_start_date)
3. **DEPT_LOCATIONS** (Dnumber, Dlocation)
4. **PROJECT** (Pnumber, Pname, Plocation, Dnum)
5. **WORKS_ON** (Essn, Pno, Hours)
6. **DEPENDENT** (Essn, Dependent_name, Sex, Bdate, Relationship)

Specifying Constraints in SQL

- NOT NULL
  - NULL is not permitted for a particular attribute
- Default value
  - **DEFAULT** <value>
- **CHECK** clause
  - Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21);
  - Score float Check (Score>=0 and Score<=10)

Specifying Constraints in SQL

- **PRIMARY KEY** clause
  - Specifies one or more attributes that make up the primary key of a relation
  - Dnumber INT PRIMARY KEY;
- **UNIQUE** clause
  - Specifies alternate (secondary) keys
  - Dname VARCHAR(15) UNIQUE;

Specifying Constraints in SQL

- **FOREIGN KEY** clause
  - Default operation: reject update on violation
  - Attach **referential triggered action** clause
    - ✓ Options include SET NULL, CASCADE, and SET DEFAULT
    - ✓ Action taken by the DBMS for SET NULL or SET DEFAULT is the same for both ON DELETE and ON UPDATE
    - ✓ CASCADE option suitable for “relationship” relations

INSERT, DELETE, and UPDATE Statements in SQL

- Three commands used to modify the database: INSERT, DELETE, UPDATE

INSERT Command in SQL

- Specify the relation name and a list of values for the tuple

INSERT INTO <Table Name> [(<List of attribute>)] **VALUES** (<List of value>)

INSERT INTO <Table Name> <Select query>

```
U1:     INSERT INTO   EMPLOYEE
        VALUES        ( 'Richard', 'K', 'Marini', '653298653', '1962-12-30', '98
                        Oak Forest, Katy, TX', 'M', 37000, '653298653', 4 );

U3B:    INSERT INTO   WORKS_ON_INFO ( Emp_name, Proj_name,
                      Hours_per_week )
        SELECT        E.Lname, P.Pname, W.Hours
        FROM          PROJECT P, WORKS_ON W, EMPLOYEE E
        WHERE         P.Pnumber=W.Pno AND W.Essn=E.Ssn;
```

DELETE Command in SQL

- Removes tuples from a relation
  - Includes a **WHERE** clause to select the tuples to be deleted

DELETE FROM <Table Name> [WHERE <Criteria>]

DELETE FROM EMPLOYEE WHERE ssn='123456789'

DELETE FROM EMPLOYEE

UPDATE Command in SQL

- Modify attribute values of one or more selected tuples
- Additional **SET** clause in the UPDATE command
  - Specifies attributes to be modified and new values

UPDATE <Table Name>

SET <Attribute 1> = <new value>,

<Attribute 2> = <new value>,…

[WHERE <Criteria>]

```
U5:     UPDATE    PROJECT
        SET       Plocation = 'Bellaire', Dnum = 5
        WHERE     Pnumber=10;
```

Content

1. Introduction
2. SQL Data Definition and Data Types
3. INSERT, DELETE, and UPDATE Statements in SQL
4. Basic Retrieval Queries in SQL

Basic Retrieval Queries in SQL

- SELECT statement
  - One basic statement for retrieving information from a database

SELECT [DISTINCT] attribute list | fucntion

FROM table list

[WHERE condition]

[GROUP BY attribute list of group]

[HAVING condition after group by]

[ORDER BY attribute 1 ASC | DESC, attribute 2 ASC | DESC,…]

[UNION | INTERSECT | MINUS <Query>]

Basic Retrieval Queries in SQL

- Basic SELECT statement

SELECT [DISTINCT] attribute list | fucntion

FROM table list

[WHERE condition]

- Attribute list: a list of attribute name whose values are to be retrieved by the query
- Table list: a list of relation names required to process the query.
- Condition: is the conditional expression that identifies the tuples to be retrieved by the query: >, <, =, AND, OR…

Figure 4.3
Results of SQL queries when applied to the COMPANY database state shown in Figure 3.6. (a) Q0. (b) Q1. (c) Q2. (d) Q8. (e) Q9. (f) Q10. (g) Q1C.

(a)

Bdate	Address
1965-01-09	731Fondren, Houston, TX

(b)

Fname	Lname	Address
John	Smith	731 Fondren, Houston, TX
Franklin	Wong	638 Voss, Houston, TX
Ramesh	Narayan	975 Fire Oak, Humble, TX
Joyce	English	5631 Rice, Houston, TX

Query 0. Retrieve the birth date and address of the employee(s) whose name is 'John B. Smith'.

```
Q0:   SELECT   Bdate, Address
      FROM     EMPLOYEE
      WHERE    Fname='John' AND Minit='B' AND Lname='Smith';
```

Query 1. Retrieve the name and address of all employees who work for the 'Research' department.

```
Q1:   SELECT   Fname, Lname, Address
      FROM     EMPLOYEE, DEPARTMENT
      WHERE    Dname='Research' AND Dnumber=Dno;
```

Figure 4.3
Results of SQL queries when applied to the COMPANY database state shown in Figure 3.6. (a) Q0. (b) Q1. (c) Q2. (d) Q8. (e) Q9. (f) Q10. (g) Q1C.

(c)

Pnumber	Dnum	Lname	Address	Bdate
10	4	Wallace	291Berry, Bellaire, TX	1941-06-20
30	4	Wallace	291Berry, Bellaire, TX	1941-06-20

Query 2. For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

```
Q2:   SELECT   Pnumber, Dnum, Lname, Address, Bdate
      FROM     PROJECT, DEPARTMENT, EMPLOYEE
      WHERE    Dnum=Dnumber AND Mgr_ssn=Ssn AND
               Plocation='Stafford';
```

Basic Retrieval Queries in SQL

- Same name can be used for two (or more) attributes
  - As long as the attributes are in different relations
  - Must **qualify** the attribute name with the relation name to prevent ambiguity

EMPLOYEE (Fname, Minit, Name, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

DEPARTMENT (Dnumber, Name, Mgr_ssn, Mgr_start_date)

Retrieve the name and address of all employees who work for the 'Research' department

```
SELECT    Fname, EMPLOYEE.Name, Address
FROM      EMPLOYEE, DEPARTMENT
WHERE     DEPARTMENT.Name = 'Research'
          AND DEPARTMENT. Dnumber = EMPLOYEE. Dno
```

Aliases

- Declare alternative relation names, attribute names: [AS] New_Name

EMPLOYEE (Fname, Minit, LName, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

DEPARTMENT (Dnumber, DName, Mgr_ssn, Mgr_start_date)

Retrieve the name and address of all employees who work for the 'Research' department

```
SELECT    Fname, E.Lname, Address
FROM      EMPLOYEE E, DEPARTMENT D
WHERE     D.Name = 'Research' AND D. Dnumber = E. Dno
SELECT    Fname + ' ' + E.Lname as FullName , Address
FROM      EMPLOYEE E, DEPARTMENT D
WHERE     D.Name = 'Research' AND D. Dnumber = E. Dno
```

Use of the Asterisk (*)

- Retrieve all the attribute values of the selected tuples: *

EMPLOYEE (Fname, Minit, LName, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

DEPARTMENT (Dnumber, DName, Mgr_ssn, Mgr_start_date)

Retrieve information of all employees who work for the 'Research' department

```
SELECT    *
FROM      EMPLOYEE E, DEPARTMENT D
WHERE     E.Name = 'Research' AND D. Dnumber = E. Dno
```

Retrieves all the attribute values of any EMPLOYEE who works in DEPARTMENT number 5

```
SELECT    *
FROM      EMPLOYEE
WHERE     Dno = 5
```

Substring Pattern Matching and Arithmetic Operators

- **LIKE** comparison operator
  - Used for string **pattern matching**
  - % replaces an arbitrary number of zero or more characters
  - underscore (_) replaces a single character
- **BETWEEN** comparison operator
- Standard arithmetic operators:
  - Addition (+), subtraction (–), multiplication (*), and division (/)

Substring Pattern Matching and Arithmetic Operators

- Retrieve all employees whose address is in Houston

EMPLOYEE (Fname, Minit, LName, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Address LIKE '%Houston%';
```

- Retrieve all employees whose salary is from 30.000 USD to 40.000 USD

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Salary BETWEEN 30000 AND 40000;
```

Substring Pattern Matching and Arithmetic Operators

- Show the resulting salaries if every employee working on the 'ProductX' project is given a 10% raise

EMPLOYEE (Fname, Minit, LName, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

PROJECT (Pnumber, Pname, Plocation, Dnum)

WORKS_ON (Essn, Pno, Hours)

```
SELECT Fname, Lname, Salary*1.1 as Increased_sal
FROM EMPLOYEE E, WORKS_ON W, PROJECT P
WHERE Ssn = Essn AND Pno = Pnumber AND Pname = 'ProductX'
```

Ordering of Query Results

- Use **ORDER BY** clause
  - Keyword **DESC** to see result in a descending order of values
  - Keyword **ASC** to specify ascending order explicitly
- Show the resulting salaries if every employee working on the 'ProductX' project is given a 10% raise.

EMPLOYEE (Fname, Minit, LName, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

PROJECT (Pnumber, Pname, Plocation, Dnum); **WORKS_ON** (Essn, Pno, Hours)

```
SELECT Fname, Lname, Salary*1.1 as Increased_sal
FROM EMPLOYEE E, WORKS_ON W, PROJECT P
WHERE Ssn = Essn AND Pno = Pnumber AND Pname = 'ProductX'
ORDER BY Lname ASC, Fname ASC
```

Tables as Sets in SQL

- SQL does not automatically eliminate duplicate tuples in query results
- Use the keyword **DISTINCT** in the SELECT clause

EMPLOYEE (Fname, Minit, LName, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

Retrieve the salary of every employee

```
SELECT    Salary
FROM      EMPLOYEE
```

```
SELECT    DISTINCT Salary
FROM      EMPLOYEE
```

Salary
30000
40000
25000
43000
38000
25000
25000
55000

Salary
30000
40000
25000
43000
38000
55000

Tables as Sets in SQL

- Set operations: UNION, **EXCEPT** (difference), **INTERSECT**
- Apply only to type- compatible relations, so we must make sure that the two relations on which we apply the operation have the same attributes and that the attributes appear in the same order in both relations.

(a)

R

A
a1
a2
a2
a3

S

A
a1
a2
a4
a5

(b)

T

A
a1
a1
a2
a2
a2
a3
a4
a5

(c)

T

A
a2
a3

(d)

T

A
a1
a2

Figure 6.5
The results of SQL multiset operations. (a) Two tables, R(A) and S(A).
(b) R(A)UNION ALL S(A).
(c) R(A) EXCEPT ALL S(A).
(d) R(A) INTERSECT ALL S(A).

Tables as Sets in SQL

- Set operations: UNION, **EXCEPT** (difference), **INTERSECT**
- Make a list of all employee that work for project number 1 or work for project number 2

EMPLOYEE (Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

PROJECT (Pnumber, Pname, Plocation, Dnum)

WORKS_ON (Essn, Pno, Hours)

```
(SELECT   Essn  FROM    PROJECT, WORKS_ON
WHERE     Pnumber = Pno AND Pno = 1)
UNION
(SELECT   Essn  FROM    PROJECT, WORKS_ON
WHERE     Pnumber = Pno AND Pno = 2)
```

Tables as Sets in SQL

- Set operations: UNION, **EXCEPT** (difference), **INTERSECT**
- Make a list of all employee that work for project number 1 and do not work for project number 2

EMPLOYEE (Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

PROJECT (Pnumber, Pname, Plocation, Dnum)

WORKS_ON (Essn, Pno, Hours)

```
(SELECT   Essn  FROM    PROJECT, WORKS_ON
WHERE     Pnumber = Pno AND Pno = 1)
EXCEPT
(SELECT   Essn  FROM    PROJECT, WORKS_ON
WHERE     Pnumber = Pno AND Pno = 2)
```

Tables as Sets in SQL

- Set operations: UNION, **EXCEPT** (difference), **INTERSECT**
- Make a list of all employee that work for both project number 1 and project number 2

EMPLOYEE (Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

PROJECT (Pnumber, Pname, Plocation, Dnum)

WORKS_ON (Essn, Pno, Hours)

```
(SELECT   Essn  FROM    PROJECT, WORKS_ON
WHERE     Pnumber = Pno AND Pno = 1)
INTERSECT
(SELECT   Essn  FROM    PROJECT, WORKS_ON
WHERE     Pnumber = Pno AND Pno = 2)
```

Tables as Sets in SQL

- Set operations: UNION, **EXCEPT** (difference), **INTERSECT**
- Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project

EMPLOYEE (Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

DEPARTMENT (Dnumber, Dname, Mgr_ssn, Mgr_start_date)

PROJECT (Pnumber, Pname, Plocation, Dnum)

WORKS_ON (Essn, Pno, Hours)

```
(SELECT   DISTINCT Pnumber  FROM   PROJECT, DEPARTMENT, EMPLOYEE
WHERE     Dnumber = Dno AND Mgr_ssn = Ssn AND Lname = 'Smith') UNION
(SELECT   DISTINCT Pnumber FROM   PROJECT, WORKS_ON, EMPLOYEE
WHERE     Pnumber = Pno AND Essn = Ssn AND Lname = 'Smith')
```

More complex SQL

- Nested queries, joined tables, outer joins, aggregate functions, and grouping

Comparisons Involving NULL and Three-Valued Logic

- Meanings of `NULL`
  - **Unknown value**
  - **Unavailable or withheld value**
  - **Not applicable attribute**
- Each individual `NULL` value considered to be different from every other `NULL` value
- SQL uses a three-valued logic:
  - `TRUE`, `FALSE`, and `UNKNOWN`

Comparisons Involving NULL and Three-Valued Logic

Table 5.1 Logical Connectives in Three-Valued Logic

(a)

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

(b)

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

(c)

NOT	
TRUE	FALSE
FALSE	TRUE
UNKNOWN	UNKNOWN

Comparisons Involving NULL and Three-Valued Logic

- SQL allows queries that check whether an attribute value is `NULL`
  - `IS` or `IS NOT NULL`

Query 18. Retrieve the names of all employees who do not have supervisors.

```
Q18:    SELECT    Fname, Lname
        FROM      EMPLOYEE
        WHERE     Super_ssn IS NULL;
```

More complex SQL

- **Nested queries**
  - Complete select-from-where blocks within WHERE clause of another query
  - **Outer query**
- Comparison operator `IN`
  - Compares value v with a set (or multiset) of values V
  - Evaluates to `TRUE` if v is one of the elements in V

More complex SQL

- **Nested queries**
  - Complete select-from-where blocks within WHERE clause of another query
  - **Outer query**

```
Q4A:  SELECT   DISTINCT Pnumber
      FROM     PROJECT
      WHERE    Pnumber IN
               ( SELECT     Pnumber
                 FROM       PROJECT, DEPARTMENT, EMPLOYEE
                 WHERE      Dnum=Dnumber AND
                            Mgr_ssn=Ssn AND Lname='Smith' )
               OR
               Pnumber IN
               ( SELECT     Pno
                 FROM       WORKS_ON, EMPLOYEE
                 WHERE      Essn=Ssn AND Lname='Smith' );
```

More complex SQL

- Use tuples of values in comparisons
  - Place them within parentheses

```
SELECT    DISTINCT Essn
FROM      WORKS_ON
WHERE     (Pno, Hours) IN ( SELECT    Pno, Hours
                            FROM      WORKS_ON
                            WHERE     Essn='123456789' );
```

More complex SQL

- Use other comparison operators to compare a single value *v*
  - = `ANY` (or = `SOME`) operator
    - Returns `TRUE` if the value *v* is equal to some value in the set *V* and is hence equivalent to `IN`
  - Other operators that can be combined with `ANY` (or `SOME`): >, >=, <, <=, and <>

More complex SQL

- Use other comparison operators to compare a single value *v*
  - = ANY (or = SOME) operator
    - Returns TRUE if the value *v* is equal to some value in the set *V* and is hence equivalent to IN
  - Other operators that can be combined with ANY (or SOME): >, >=, <, <=, and <>

```
SELECT    Lname, Fname
FROM      EMPLOYEE
WHERE     Salary > ALL   ( SELECT    Salary
                           FROM      EMPLOYEE
                           WHERE     Dno=5 );
```

More complex SQL

- Avoid potential errors and ambiguities
  - Create tuple variables (aliases) for all tables referenced in SQL query

Query 16. Retrieve the name of each employee who has a dependent with the same first name and is the same sex as the employee.

```
Q16:    SELECT    E.Fname, E.Lname
        FROM      EMPLOYEE AS E
        WHERE     E.Ssn IN  ( SELECT    Essn
                              FROM      DEPENDENT AS D
                              WHERE     E.Fname=D.Dependent_name
                                        AND E.Sex=D.Sex );
```

More complex SQL

- `EXISTS` function
  - Check whether the result of a correlated nested query is empty or not
- `EXISTS` and `NOT EXISTS`
  - Typically used in conjunction with a correlated nested query
- SQL function `UNIQUE(Q)`
  - Returns `TRUE` if there are no duplicate tuples in the result of query Q

More complex SQL

- Retrieve employees who work on **all** the projects

EMPLOYEE (Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

PROJECT (Pnumber, Pname, Plocation, Dnum)

WORKS_ON (Essn, Pno, Hours)

```
SELECT *
FROM EMPLOYEE e
WHERE NOT EXITST (SELECT *
                  FROM PROJECT p
                  WHERE NOT EXITST (SELECT *
                                    FROM WORKS_ON w
                                    WHERE w.Essn = e.Ssn and w.Pno = Pnumber))
```

Joined Tables in SQL and Outer Joins

- **Joined table**
  - Permits users to specify a table resulting from a join operation in the FROM clause of a query
- The FROM clause in Q1A
  - Contains a single joined table

```
Q1A:    SELECT    Fname, Lname, Address
        FROM      (EMPLOYEE JOIN DEPARTMENT ON Dno=Dnumber)
        WHERE     Dname='Research';
```

Joined Tables in SQL and Outer Joins

- **Inner join**
  - Default type of join in a joined table
  - Tuple is included in the result only if a matching tuple exists in the other relation
- LEFT OUTER JOIN
  - Every tuple in left table must appear in result
  - If no matching tuple: Padded with NULL values for attributes of right table
- RIGHT OUTER JOIN
  - Every tuple in right table must appear in result
  - If no matching tuple: Padded with NULL values for the attributes of left table
- FULL OUTER JOIN

Joined Tables in SQL and Outer Joins

- Retrieve employees (Ssn, Fname) that do not work on any project

EMPLOYEE (Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

WORKS_ON (Essn, Pno, Hours)

```
SELECT Ssn, Fname
FROM EMPLOYEE e left outer join WORKS_ON w on e.Ssn = w.Essn
WHERE Pno IS NULL
```

Aggregate Functions in SQL

- Used to summarize information from multiple tuples into a single-tuple summary
- **Grouping**
  - Create subgroups of tuples before summarizing
- Built-in aggregate functions
  - **`COUNT`**, **`SUM`**, **`MAX`**, **`MIN`**, and **`AVG`**
- Functions can be used in the `SELECT` clause or in a `HAVING` clause

Aggregate Functions in SQL

- NULL values discarded when aggregate functions are applied to a particular column

Query 20. Find the sum of the salaries of all employees of the 'Research' department, as well as the maximum salary, the minimum salary, and the average salary in this department.

```
Q20:   SELECT   SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)
       FROM     (EMPLOYEE JOIN DEPARTMENT ON Dno=Dnumber)
       WHERE    Dname='Research';
```

Aggregate Functions in SQL

- NULL values discarded when aggregate functions are applied to a particular column

Queries 21 and 22. Retrieve the total number of employees in the company (Q21) and the number of employees in the 'Research' department (Q22).

```
Q21:    SELECT    COUNT (*)
        FROM      EMPLOYEE;

Q22:    SELECT    COUNT (*)
        FROM      EMPLOYEE, DEPARTMENT
        WHERE     DNO=DNUMBER AND DNAME='Research';
```

Group by

- **Partition** relation into subsets of tuples
  - Based on **grouping attribute(s)**
  - Apply function to each such group independently
- **`GROUP BY`** clause
  - Specifies grouping attributes
- If NULLs exist in grouping attribute
  - Separate group created for all tuples with a NULL value in grouping attribute

Group by

- For each department, retrieve the department number and the number of its employees

EMPLOYEE (Fname, Minit, LName, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

DEPARTMENT (Dnumber, Dname, Mgr_ssn, Mgr_start_date)

```
SELECT Dno, count (Ssn) FROM EMPLOYEE
GROUP BY Dno
```

- For each department, retrieve the department number, department name and the number of its employees

```
SELECT Dno, Dname, count (Ssn)
FROM EMPLOYEE e join DEPARTMENT d on e.Dno = d.Dnumber
GROUP BY Dno, Dname
```

Having

- Provides a condition on the summary information
- For each department that has more than 5 employees , retrieve the department number, department name and the number of its employees

```
SELECT Dno, Dname, count (Ssn)
FROM EMPLOYEE e join DEPARTMENT d on e.Dno = d.Dnumber
GROUP BY Dno, Dname
HAVING count(Ssn)>5
```

References

1. Ramez Elmasri, and Shamkant B. Navathe, *Fundamentals of Database Systems (7th Ed.)*, Addison-Wesley, 2016.

2. Nguyen Dang Ty, and Do Phuc, *Database*, Vietnam National University – HCMC, 2015.

3. George Tillmann, *Usage-Driven Database Design,* Apress, 2017.

Q A